

AGEX Development Roadmap

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX Product Roadmap을 실제 개발 수행 단계로 전환하기 위한 공식 개발 로드맵이다.

Product Roadmap이 “어떤 제품 상태로 발전할 것인가”를 정의한다면, Development Roadmap은 이를 구현하기 위해 어떤 기술 기반과 개발 단계를 어떤 순서로 구축해야 하는지를 정의한다.

본 문서는 이후 작성되는 다음 문서의 상위 실행 기준으로 사용한다.

- WBS
- Development Plan
- Resource Plan
- Cost Estimation
- Release Plan
- Test Plan
- Operations Preparation

1.2 적용 범위

본 문서는 다음 개발 영역을 포함한다.

- Core Platform
- API
- Runtime
- Agent Framework
- Workflow Engine
- Knowledge Engine
- Memory Engine
- Plugin Framework

- Model Gateway
- Marketplace
- SDK / CLI
- Security
- Multi-Tenancy
- Governance
- Deployment
- Observability
- DevOps
- QA
- Documentation

1.3 Development Roadmap의 공식 정의

AGEX Development Roadmap은 Product Architecture와 Product Roadmap에서 정의한 AI Operating System을 실제 개발 가능한 Workstream과 Release 단위로 분해하고, 기술 의존성·검증 순서·품질 Gate를 기준으로 단계적으로 구현하기 위한 공식 실행 로드맵이다.

2. 개발 로드맵 기본 원칙

AGEX 개발은 다음 원칙을 따른다.

- Architecture Before Coding
- Contract Before Implementation
- Runtime Before Advanced Autonomy
- Tenant Isolation from Day One
- Security from Day One
- Observability from Day One
- API First

- Schema First
 - Test Before Promotion
 - Immutable Published Resource
 - Provider Abstraction
 - Modular Monolith Before Premature Microservices
 - Continuous Integration
 - Reproducible Build
 - Incremental Delivery
 - Backward Compatibility
 - Feature Flag for Risky Rollout
 - Stage Gate Based Development
 - No Hidden Production Shortcut
-

3. 개발 단계 전체 구조

AGEX 개발은 다음 8개 Development Phase로 구분한다.

Phase 0

Development Foundation



Phase 1

Core Platform Foundation



Phase 2

Runtime & Execution Core



Phase 3

Agent / Workflow Core



Phase 4

Knowledge / Memory / Plugin



Phase 5

Enterprise Control Layer



Phase 6

Marketplace & Ecosystem



Phase 7

Advanced Autonomous Platform

각 Phase는 이전 단계의 핵심 기반을 전제로 한다.

4. Phase 0 — Development Foundation

4.1 목표

코딩이 본격적으로 시작되기 전에 개발 구조, Repository, 표준, 테스트, CI/CD 및 기본 Architecture Contract를 확정한다.

4.2 핵심 작업

- Repository 구조 정의
- Branch Strategy
- Coding Convention
- API Convention
- Resource Schema Convention

- Error Code Convention
- Logging Convention
- Event Convention
- Test Strategy
- Environment Strategy
- Local Development Environment
- CI Pipeline
- Dependency Management
- Security Scan
- Artifact Versioning
- Container Build Standard

4.3 Repository 구조

초기에는 다음과 같은 Monorepo 또는 명확한 Multi-Repo 전략 중 하나를 확정한다.

예시:

agex/

├─ apps/

| └─ api

| └─ console

| └─ worker

|

├─ packages/

| └─ core

| └─ runtime

| └─ agent

| └─ workflow

```
|   ├── knowledge
|   ├── memory
|   ├── plugin
|   ├── model
|   └── sdk
|
|── schemas/
|── infrastructure/
|── tests/
└── docs/
```

실제 구조는 기술 Stack 결정 후 확정한다.

4.4 Phase 0 완료 기준

1. 개발 Repository가 준비된다.
2. Local 실행이 가능하다.
3. CI가 자동 실행된다.
4. 기본 Unit Test Framework가 동작한다.
5. Container Build가 가능하다.
6. Security Scan이 연결된다.
7. API 및 Schema Convention이 확정된다.
8. Version 규칙이 확정된다.
9. 개발 환경과 Production 환경의 Configuration 분리 원칙이 확정된다.
10. 초기 Architecture Decision Record 체계가 준비된다.

5. Phase 1 — Core Platform Foundation

5.1 목표

AGEX의 모든 Resource와 Tenant를 관리할 수 있는 최소 Platform Core를 구축한다.

핵심 개발 대상:

- Tenant
 - Identity Context
 - Resource Registry
 - Resource Lifecycle
 - Version / Revision
 - Configuration
 - Policy Baseline
 - Audit
 - Usage
 - API Gateway
-

6. Tenant Core 개발

6.1 구현 항목

- Tenant Resource
- Tenant Lifecycle
- Tenant Context
- Tenant Resolution
- Tenant Middleware
- Tenant Membership
- Tenant Role
- Tenant Configuration
- Tenant Quota

6.2 필수 테스트

- Tenant A → Tenant B Resource 접근 차단
 - Tenant Context 누락 차단
 - Token Tenant 불일치 차단
 - Tenant Suspension 반영
 - Tenant Resource Query Isolation
-

7. Resource Registry 개발

7.1 구현 항목

- Resource ID
- Resource Type
- Tenant Ownership
- Resource Metadata
- Version
- Revision
- Lifecycle
- Labels
- Checksum

7.2 기본 API

create

get

list

update

delete

publish

activate

suspend

deprecate

7.3 Published Resource

Published Resource는 Immutable Version으로 처리한다.

8. Schema Registry 개발

8.1 목적

모든 주요 Resource, Event, API Contract를 공식 Schema로 관리한다.

8.2 구현 항목

- Schema ID
- Schema Version
- Validation
- Compatibility Check
- Schema Diff
- Schema Registry API

8.3 활용

- API Validation
 - SDK Generation
 - Event Validation
 - Plugin Validation
-

9. API Gateway 기본 구현

구현 기능:

- Routing
- Authentication Integration

- Tenant Resolution
- Request ID
- Basic Rate Limit
- Request Size Limit
- API Version Routing
- Error Normalization

초기 Gateway에 복잡한 Domain Logic을 구현하지 않는다.

10. Policy Baseline 개발

초기 Policy Engine은 최소 다음 기능을 제공한다.

- Permit / Deny
- Tenant Scope
- Role
- Resource
- Action
- Basic Attribute
- Policy Version

복잡한 Governance Rule은 이후 확장한다.

11. Audit Core 개발

구현 항목:

- Audit Event
- Principal
- Tenant
- Resource

- Action
- Result
- Request ID
- Trace ID
- Timestamp

초기 필수 Audit 대상:

- Authentication
- Resource Publish
- Permission Change
- Tenant Change
- Policy Change

12. Usage Core 개발

초기부터 Usage 측정 구조를 포함한다.

대상:

- API
- Execution
- Model
- Plugin
- Storage

초기에는 과금 기능이 없어도 Tenant별 Usage Attribution은 가능해야 한다.

13. Phase 1 데이터베이스 전략

초기 Transactional Store에는 다음을 저장한다.

- Tenant

- Resource Registry
- Configuration
- Policy
- Audit Metadata
- Usage Metadata

초기에는 단일 관계형 Database를 사용해도 되지만 Domain별 Repository Boundary를 유지한다.

14. Phase 1 완료 Gate

다음이 모두 가능해야 한다.

Tenant 생성

→ 사용자 Context 연결

→ Resource 생성

→ Resource Version 관리

→ Resource Publish

→ Permission 검증

→ Audit 조회

기술 조건:

- Tenant Isolation Test Pass
 - API Contract Test Pass
 - Database Migration 자동화
 - Basic Observability
 - CI/CD 운영 가능
-

15. Phase 2 — Runtime & Execution Core

15.1 목표

AGEX에서 모든 AI와 Workflow 실행을 관리하는 Durable Runtime을 구축한다.

핵심 Component:

- Execution Manager
- Scheduler
- Dispatcher
- Queue
- Worker
- Execution State
- Task State
- Retry
- Timeout
- Cancellation
- Checkpoint
- Recovery

16. Execution Resource 구현

필드:

- Execution ID
- Tenant ID
- Target Resource
- Target Version
- Execution Type
- State
- Priority

- Input Reference
- Output Reference
- Created At
- Started At
- Completed At
- Timeout
- Budget
- Trace ID

17. Execution State Machine 구현

최소 State:

Created

→ Validating

→ Queued

→ Running

→ Waiting

→ Retrying

→ Completed

→ Failed

→ Cancelled

→ TimedOut

Transition은 Domain Logic으로 강제한다.

Database의 상태 값을 임의 Update하는 방식으로 관리하지 않는다.

18. Task Runtime 구현

Execution 내부 작업을 Task 단위로 분리한다.

Task 필드:

- Task ID
- Execution ID
- Tenant ID
- Task Type
- State
- Attempt
- Input
- Output
- Worker
- Lease
- Timeout

19. Queue 개발

초기 Queue 요구사항:

- At-Least-Once Delivery
- Ack
- Visibility / Lease
- Retry Queue
- DLQ
- Tenant Context

Queue를 Execution State Source of Truth로 사용하지 않는다.

20. Worker 개발

초기 Worker 요구사항:

- Stateless
 - Worker Identity
 - Tenant Context Validation
 - Task Lease
 - Heartbeat
 - Graceful Shutdown
 - Cancellation
 - Trace
 - Resource Limit
-

21. Retry Engine 개발

지원 항목:

- Max Attempts
- Backoff
- Jitter
- Retryable Error
- Retry Budget
- Idempotency

Write Side Effect가 있는 Task는 자동 Retry 정책을 별도로 제한한다.

22. Timeout Engine 개발

구현 대상:

- Execution Timeout
- Task Timeout

- External Call Timeout
- Wait Timeout

Timeout은 Worker Local Timer에만 의존하지 않는다.

23. Cancellation 개발

Cancellation 흐름:

Cancel Request

- Execution State Update
 - Stop New Task Dispatch
 - Worker Signal
 - Checkpoint / Cleanup
 - Cancelled
-

24. Checkpoint 개발

Checkpoint 대상:

- Workflow Step
- Agent Loop
- Long Running Task
- External Action Boundary

Checkpoint 저장 내용은 재시작에 필요한 최소 실행 상태로 한다.

25. Recovery Engine 개발

Recovery Flow:

Worker Failure

- Lease Expire

- State Check
 - Checkpoint Load
 - Idempotency Check
 - Redispatch
-

26. Runtime Event 개발

최소 Event:

- ExecutionCreated
 - ExecutionStarted
 - TaskQueued
 - TaskStarted
 - TaskCompleted
 - ExecutionCompleted
 - ExecutionFailed
 - ExecutionCancelled
-

27. Runtime Observability

필수 Metric:

- Execution Count
- Execution Success
- Failure Rate
- Queue Depth
- Running Tasks
- Retry
- Timeout

- Worker Health
 - Execution Duration
-

28. Runtime 테스트

필수 테스트:

- Worker Kill Recovery
 - Queue Redelivery
 - Duplicate Task
 - Cancellation
 - Timeout
 - Retry
 - Database Restart
 - Checkpoint Recovery
 - Tenant Context Preservation
-

29. Phase 2 완료 Gate

다음 흐름이 안정적으로 동작해야 한다.

Execution 생성

→ Queue

→ Worker

→ Task 실행

→ Worker 장애

→ Recovery

→ 결과 완료

필수 기준:

- Durable Execution
 - Duplicate Safety
 - Cancellation
 - Timeout
 - Recovery
 - Audit
 - Trace
-

30. Phase 3 — Agent / Workflow Core

30.1 목표

Runtime 위에서 AGEX의 핵심 AI 실행 주체인 Agent와 Workflow를 구현한다.

31. Agent Definition 개발

구현 항목:

- Agent Resource
 - Goal
 - Instruction
 - Capability
 - Permission
 - Model Policy
 - Knowledge Scope
 - Memory Policy
 - Tool Scope
 - Autonomy Level
-

32. Agent Runtime 개발

Agent Loop:

Observe

→ Build Context

→ Evaluate

→ Plan

→ Select Action

→ Validate

→ Execute

→ Observe Result

→ Update

→ Complete

33. Agent Action 모델 구현

초기 Action:

- Generate
- Call Model
- Call Tool
- Retrieve Knowledge
- Read Memory
- Ask Approval
- Complete
- Fail

Stage 확장 Action:

- Delegate

- Start Workflow
 - Call Agent
-

34. Context Builder 구현

Context Source:

- System Policy
- Agent Definition
- User Task
- Runtime State
- Knowledge
- Memory
- Tool Result

34.1 Priority

상위 Security 및 Policy Instruction과 Retrieved Content를 명확히 분리한다.

35. Model Gateway 초기 구현

핵심 Component:

- Model Registry
- Model Provider Interface
- Provider Adapter
- Model Router
- Model Usage
- Provider Health

초기 Provider는 소수만 구현하더라도 Contract는 범용으로 설계한다.

36. Model Request Contract

공통 입력:

- Model Capability
- Messages / Input
- Structured Output Schema
- Tool Definitions
- Token Limit
- Temperature Policy
- Streaming
- Tenant
- Data Classification

공통 출력:

- Content
- Tool Call
- Structured Output
- Usage
- Provider
- Model
- Latency

37. Model Routing 초기 규칙

초기에는 다음 기준만으로도 시작할 수 있다.

Required Capability

→ Allowed Provider

→ Tenant Policy

→ Model Availability

→ Selected Model

비용·품질 기반 동적 Routing은 이후 확장한다.

38. Workflow Definition 개발

구현 항목:

- Workflow Resource
- Input Schema
- Output Schema
- Step
- Transition
- Variable
- Retry
- Timeout
- Error Policy

39. 초기 Workflow Step

Phase 3 최소 지원:

- Agent Step
- Model Step
- Function Step
- Plugin Placeholder Step
- Condition
- Wait
- Approval

- End

40. Workflow Instance 개발

구현 항목:

- Workflow Instance
- Workflow Version
- Step State
- Variable State
- Execution Mapping
- Checkpoint

41. Workflow Engine 개발

초기 Execution Flow:

Workflow Start

→ Resolve Step

→ Schedule

→ Execute

→ Evaluate Transition

→ Next Step

→ Complete

42. Human Approval 개발

Approval Resource:

- Approval ID
- Execution

- Requested Action
- Risk
- Payload Digest
- Requested By
- Approver
- State
- Expiration

Approval 결과:

- Approved
- Rejected
- Expired

43. Agent/Workflow 테스트

필수:

- Model failure
- Structured output invalid
- Tool permission denied
- Approval rejected
- Agent timeout
- Workflow resume
- Invalid transition
- Workflow cancellation
- Version pinning

44. Phase 3 완료 Gate

다음이 실제로 가능해야 한다.

Agent 생성

→ Publish

→ Model 연결

→ 실행

→ Approval

→ 결과

Workflow 생성

→ Agent Step

→ Condition

→ Approval

→ Complete

45. Phase 4 — Knowledge / Memory / Plugin

45.1 목표

Agent와 Workflow에 지속적인 정보·기억·외부 실행 능력을 제공한다.

46. Knowledge Engine 개발

1차 구현 우선순위:

1. Knowledge Source
2. Collection
3. Ingestion
4. Parser
5. Chunking

6. Metadata
 7. Embedding
 8. Vector Index
 9. Keyword Index
 10. Hybrid Retrieval
 11. Citation
-

47. Knowledge Ingestion Worker

별도 비동기 Worker로 구현한다.

Pipeline:

Fetch

- Parse
 - Normalize
 - Classify
 - Chunk
 - Embed
 - Index
 - Verify
-

48. Knowledge Retrieval 개발

Query Flow:

Tenant

- Permission
- Collection
- Query

→ Keyword Search

→ Vector Search

→ Merge

→ Rank

→ Citation

49. Knowledge 테스트

- Tenant isolation
- permission filtering
- deleted document removal
- duplicate document
- malformed document
- retrieval relevance
- citation correctness
- reindex

50. Memory Engine 초기 개발

1차 구현:

- Working Memory
- Session Memory
- Agent Memory
- Memory Proposal
- Memory Write Policy
- Memory Retrieval
- Retention

- Expiration
 - Deletion
-

51. Memory Store

초기 Storage:

- Transactional Memory Metadata
- Vector Index
- Cache for Working/Session

Memory Source of Truth와 Vector Index를 분리한다.

52. Memory Write Pipeline

Candidate

- Scope
 - Classification
 - Permission
 - Duplicate Check
 - Retention
 - Persist
-

53. Memory Retrieval Pipeline

Query

- Tenant
- Scope
- Permission
- Search

→ Rank

→ Token Budget

→ Context

54. Plugin Framework 개발

우선 구현:

- Plugin Manifest
- Plugin Registry
- Plugin Installation
- Plugin Action
- Input Schema
- Output Schema
- Permission
- Secret Binding
- Plugin Gateway

55. Plugin Runtime

초기에는 다음 2가지 형태로 시작할 수 있다.

- Built-in Trusted Adapter
- Isolated External Plugin Runtime

이후 Trust Level별 Sandbox를 강화한다.

56. Plugin Credential Broker

Plugin 실행 시:

Action Request

- Permission
- Credential Reference
- Credential Broker
- Limited Credential
- Plugin Runtime

Agent에게 Credential을 전달하지 않는다.

57. Plugin Side Effect

Action 분류:

- Read Only
- Reversible Write
- Irreversible Write
- Privileged Action

이 분류를 Workflow 및 Approval과 연동한다.

58. Phase 4 완료 Gate

다음 통합 시나리오가 동작해야 한다.

Agent

- Knowledge Retrieval
- Memory Recall
- Model
- Plugin Action
- Memory Proposal
- Complete

그리고:

Workflow

→ Agent

→ Plugin

→ Approval

→ External Action

→ Complete

59. Phase 5 — Enterprise Control Layer

59.1 목표

AGEX를 Enterprise 운영 수준으로 확장한다.

핵심 영역:

- Advanced Security
 - Governance
 - Dedicated Tenant
 - Private Deployment
 - Cost Control
 - Advanced Observability
-

60. Advanced Authorization

추가 구현:

- ABAC
- Policy Conditions
- Resource Attributes
- Environment Context
- Data Classification

- Risk Context

61. Governance Engine

구현 항목:

- Risk Resource
- Approval Policy
- Governance Rule
- Review
- Exception
- Evaluation Gate
- Change Request

62. Risk Engine

Risk 평가:

- Permission
- Data
- Side Effect
- Autonomy
- External Network
- Cost
- Recoverability

Risk가 Approval 및 Runtime Admission에 영향을 주도록 한다.

63. Policy Simulation

새 Policy 활성화 전에 영향도를 분석한다.

기능:

- affected resources
 - newly denied actions
 - newly allowed actions
 - model routing changes
-

64. Model Governance 개발

구현 항목:

- Provider Trust
 - Model Approval
 - Model Evaluation
 - Data Policy
 - Region Policy
 - Cost Policy
 - Model Deprecation
-

65. Advanced Model Router

추가 Routing 기준:

- Capability
- Data Classification
- Security
- Quality
- Latency
- Cost
- Health

- Region
-

66. Advanced Memory

추가 구현:

- Episodic Memory
 - Semantic Memory
 - Procedural Memory
 - Consolidation
 - Conflict Resolution
 - Confidence
 - Forgetting
-

67. Advanced Knowledge

추가 구현:

- Connector Framework
 - Incremental Sync
 - Structured Retrieval
 - Re-ranking
 - MMR
 - Entity
 - Graph optional
 - Freshness
-

68. Multi-Agent 개발

구현 항목:

- Delegation
- Child Execution
- Agent Message
- Role
- Collaboration
- Delegation Depth Limit

Parent Permission을 Child Agent에 자동 상속하지 않는다.

69. Advanced Workflow

추가 Step:

- Parallel
 - Join
 - Loop
 - Event Wait
 - Sub-Workflow
 - Compensation
 - Dynamic Step
-

70. Dedicated Tenant

구현:

- Tenant Placement
- Storage Routing
- Dedicated DB Profile
- Dedicated Worker Pool
- Dedicated Runtime Profile

71. Private Deployment

개발 대상:

- Deployment Package
- Private Runtime Connector
- On-Prem Configuration
- Local Secret Adapter
- Local Model Adapter
- Offline Package Support

72. Enterprise Observability

추가:

- SLA Dashboard
- Security Dashboard
- Cost Dashboard
- Tenant Dashboard
- Model Analytics
- Plugin Analytics
- Agent Analytics

73. Cost Engine

구현:

- Usage Aggregation
- Model Cost
- Plugin Cost

- Compute Cost
 - Budget
 - Cost Alert
 - Tenant Attribution
-

74. Phase 5 완료 Gate

다음 기준을 충족해야 한다.

- L3 안정적 운영
 - L4 제한적 운영 가능
 - Advanced Governance
 - Private Deployment
 - Dedicated Tenant
 - Data Residency
 - Cost Governance
 - Enterprise Security
-

75. Phase 6 — Marketplace & Ecosystem

75.1 목표

AGEX Extension 생태계를 외부 Publisher에게 개방한다.

76. Package Format 안정화

Package 유형:

- Plugin
- Agent
- Workflow

- Knowledge
- Composite

공통 Package Manifest를 확정한다.

77. Publisher Registry

구현:

- Publisher Identity
 - Verification
 - Signing Key
 - Status
 - Trust
-

78. Submission Pipeline

Submit

- Manifest Validation
 - Signature
 - Dependency
 - Security Scan
 - Contract Test
 - Runtime Test
 - Review
 - Publish
-

79. Marketplace Catalog

기능:

- Search
 - Category
 - Capability
 - Trust
 - Version
 - Compatibility
 - Publisher
 - Installation
-

80. Package Installation Engine

구현:

- Dependency Resolver
 - Permission Diff
 - Compatibility
 - Configuration
 - Credential Binding
 - Activation
-

81. Marketplace Security

구현:

- Malware Scan
- Vulnerability Scan
- SBOM
- Signature Verification
- Package Block

- Publisher Suspend

82. Marketplace Commercial Layer

추후 순서로 구현:

- Pricing
- License
- Subscription
- Billing
- Settlement
- Rating
- Review

기술 Marketplace 안정화가 상업 기능보다 먼저다.

83. Private Marketplace

Enterprise Tenant용:

- Internal Package
- Approved Catalog
- Tenant Publisher
- Private Distribution

84. Phase 6 완료 Gate

다음이 가능해야 한다.

External Developer

→ SDK

→ Package

- Sign
 - Submit
 - Validate
 - Marketplace
 - Tenant Install
 - Runtime Execute
-

85. Phase 7 — Advanced Autonomous Platform

85.1 목표

Governance 범위 안에서 AGEX의 자율 실행 능력을 확대한다.

86. L4 실행 고도화

Continuous Policy-Based Execution 구현:

- Long-running Agent
 - Scheduled Agent
 - Event-driven Agent
 - Continuous Evaluation
 - Budget
 - Kill Switch
-

87. L5 Multi-Agent

개발:

- Agent Team
- Dynamic Agent Selection
- Role Assignment

- Negotiation
 - Shared Goal
 - Collaborative Plan
 - Team Evaluation
-

88. Dynamic Workflow Composition

AI가 Workflow Candidate를 생성할 수 있도록 한다.

필수 제한:

- registered step types
 - permission
 - policy
 - maximum depth
 - loop limit
 - cost limit
-

89. Dynamic Capability Discovery

Agent가 Registry에서 Capability를 검색한다.

Need

→ Capability Query

→ Registry

→ Candidate

→ Trust

→ Permission

→ Select

설치 권한과 Discovery는 분리한다.

90. Autonomous Optimization

다음 Candidate를 자동 생성할 수 있다.

- Model routing optimization
- Prompt improvement
- Tool selection improvement
- Workflow optimization
- Resource allocation

자동 Production 적용은 하지 않는다.

91. Continuous Evaluation Platform

모든 주요 Execution에서 품질 평가 데이터를 축적한다.

구현:

- Evaluation Queue
 - Evaluation Worker
 - Benchmark
 - Regression
 - Quality Trend
 - Cost-adjusted Quality
-

92. Self-Improvement Candidate Pipeline

Operational Data

→ Pattern Detection

→ Improvement Candidate

→ Test

- Evaluation
- Governance
- Publish

93. Development Workstream 구조

전체 개발은 다음 Workstream으로 병행 관리한다.

WS-01 Core Platform

WS-02 Runtime

WS-03 Agent

WS-04 Workflow

WS-05 Knowledge

WS-06 Memory

WS-07 Plugin

WS-08 Model Gateway

WS-09 Marketplace

WS-10 SDK / CLI

WS-11 Security

WS-12 Multi-Tenant

WS-13 Governance

WS-14 Deployment / DevOps

WS-15 Observability

WS-16 QA / Evaluation

WS-17 Developer Console

WS-18 Documentation

94. Workstream Dependency

핵심 선후관계:

Core

↓

Runtime

↓

Agent / Workflow

↓

Knowledge / Memory / Plugin

↓

Governance / Enterprise

↓

Marketplace

↓

Autonomous Platform

Model Gateway는 Runtime-Agent와 병행 구축한다.

Security, Multi-Tenant, Observability는 전체 기간에 걸쳐 병행한다.

95. 개발 병렬화 기준

다음 Workstream은 일정 구간부터 병행 가능하다.

예:

Runtime Core

├─ Agent Framework

├─ Workflow Engine

└─ Model Gateway

그리고:

Agent / Workflow Contract 안정화

└─ Knowledge

└─ Memory

└─ Plugin

96. 병렬화 금지 영역

다음은 기반 없이 선행해서는 안 된다.

- Public Marketplace before Plugin Contract
- Multi-Agent before Execution Runtime
- L4/L5 before Governance
- Dedicated Tenant before Tenant Routing
- Advanced Model Routing before Model Registry
- SDK Generation before Schema Contract

97. Development Release 단위

각 Phase는 큰 단위로 한 번에 Release하지 않는다.

권장:

Milestone

→ Increment

→ Release Candidate

→ Validation

→ Release

예:

Phase 2 Runtime:

- R2.1 Execution Resource
 - R2.2 Queue/Worker
 - R2.3 Retry/Timeout
 - R2.4 Checkpoint/Recovery
-

98. Feature Completion 정의

기능은 Code Merge만으로 완료로 보지 않는다.

Definition of Done:

1. Code
 2. Unit Test
 3. Contract Test
 4. Security Review where required
 5. Logging
 6. Metrics
 7. Error Handling
 8. API/Schema
 9. Documentation
 10. Acceptance Test
-

99. Technical Definition of Done

Core Component는 추가로 다음을 요구한다.

- Migration Strategy
- Rollback
- Load Test
- Failure Test

- Tenant Isolation
 - Trace
 - Compatibility
-

100. API Development Strategy

API는 다음 순서로 구현한다.

Schema

→ API Contract

→ Mock

→ Client Test

→ Server Implementation

→ Contract Test

→ SDK

Server 구현 후 문서를 작성하는 방식을 기본으로 하지 않는다.

101. Database Development Strategy

Database 중심 설계보다 Domain Resource 중심 설계를 사용한다.

순서:

Domain Contract

→ Resource Model

→ State Transition

→ Repository Contract

→ Physical Schema

102. Event Development Strategy

Event는 다음이 먼저 정의되어야 한다.

- Event Name
- Producer
- Consumer
- Schema
- Version
- Delivery Semantics
- Idempotency

코드에서 임의 Event 이름을 생성하지 않는다.

103. Security Development Strategy

Security는 별도 마지막 Phase가 아니다.

각 Feature 개발 시 동시에 적용한다.

예:

Agent 개발 시:

- Agent permission
- model data policy
- tool validation
- audit

Plugin 개발 시:

- secret
 - network
 - sandbox
 - output validation
-

104. Security Gate

Production Promotion 전 최소 검사:

- Static Scan
- Dependency Scan
- Secret Scan
- Auth Test
- Tenant Isolation
- Permission Test
- Input Validation

고위험 Component:

- Penetration Test
- Threat Model
- Sandbox Test

105. Observability Development Strategy

모든 Component에서 최소 다음을 동시에 구현한다.

- Log
- Metric
- Trace
- Health

"기능 개발 후 모니터링 추가" 방식으로 미루지 않는다.

106. QA Strategy

QA는 다음 계층으로 구성한다.

- Unit

- Component
 - Contract
 - Integration
 - E2E
 - Security
 - Performance
 - Chaos
 - Evaluation
-

107. AI Evaluation Strategy

일반 Software Test와 AI Evaluation을 분리한다.

Software Test:

- deterministic correctness

AI Evaluation:

- quality
- relevance
- groundedness
- tool selection
- policy compliance

둘 다 통과해야 한다.

108. Test Data Strategy

Test Data는 다음 원칙을 따른다.

- Synthetic First
- No Unnecessary Production Data

- Tenant-separated fixtures
 - Versioned Evaluation Set
 - Sensitive Data Controls
-

109. Performance Development

초기부터 주요 SLO 후보 지표를 측정한다.

- API Latency
- Execution Start Latency
- Queue Wait
- Model Gateway Latency
- Knowledge Retrieval Latency
- Workflow Duration

정확한 SLO 수치는 운영 검증 후 확정한다.

110. Scalability Test 단계

단계 1

단일 Tenant / 소규모 실행.

단계 2

다수 Tenant / 동시 실행.

단계 3

대규모 Knowledge 및 Memory.

단계 4

GPU / Private Model.

단계 5

Marketplace Extension 대량 실행.

111. CI Pipeline

기본 CI:

Checkout

→ Dependency Install

→ Lint

→ Unit Test

→ Schema Validation

→ Contract Test

→ Security Scan

→ Build

→ Container Build

→ SBOM

112. CD Pipeline

기본 CD:

Artifact

→ Development

→ Integration Test

→ Staging

→ E2E

→ Performance / Security Gate

→ Approval

→ Production

113. Branch Strategy

구체 전략은 조직 개발 방식에 따라 결정하되 다음 원칙을 유지한다.

- main branch deployable
 - short-lived feature branch
 - PR review
 - protected release branch/tag
 - no direct production modification
-

114. Release Version

Platform Version 외에 다음 Version을 추적한다.

- API
- Schema
- Runtime
- Agent Contract
- Workflow Contract
- Plugin Manifest
- SDK

모든 Version이 같은 숫자를 가질 필요는 없다.

115. Backward Compatibility Development

변경 시 반드시 다음을 확인한다.

- 기존 Agent
- 기존 Workflow
- Plugin Package
- SDK

- API Client
 - Long-running Execution
-

116. Migration Framework

필요한 Migration 유형:

- DB Schema
- Resource Definition
- Configuration
- Package Manifest
- Index
- Memory
- Knowledge

Migration은 Version과 상태를 기록해야 한다.

117. Feature Flag Strategy

고위험 신규 기능:

- New Runtime
- New Router
- New Retrieval
- New Agent Planner

은 Feature Flag로 단계적 적용할 수 있다.

Security baseline을 Disable하는 Flag는 만들지 않는다.

118. Technical Debt Management

Debt 유형:

- temporary adapter
- duplicated contract
- deprecated API
- non-scalable repository
- hard-coded provider
- local-only implementation

각 Debt에는:

- Owner
- Severity
- Removal Phase

를 지정한다.

119. Architecture Decision Record

중요 기술 결정은 ADR로 기록한다.

예:

- primary language
- database
- queue
- orchestration
- vector store
- event infrastructure
- API protocol

ADR은 Architecture 문서의 원칙을 바꾸는 것이 아니라 구체 구현 결정을 기록한다.

120. Technology Selection Gate

기술 선택 기준:

- Contract Compatibility
- Scale
- Security
- Operational Complexity
- Portability
- Cost
- Team Capability
- Ecosystem Maturity

유행이나 단순 개발 편의만으로 핵심 기술을 선택하지 않는다.

121. Initial Technology Decisions

Development Plan 작성 시 다음을 최종 확정해야 한다.

- Primary Backend Language
- Web Framework
- RDBMS
- Queue
- Cache
- Object Storage
- Vector Store
- Search Engine
- Container Platform
- Observability Stack
- Secret Store
- Model Runtime

Development Roadmap에서는 특정 Vendor를 최종 고정하지 않는다.

122. Developer Console Roadmap

초기

- Tenant
- Agent
- Workflow
- Execution
- Model

중기

- Knowledge
- Memory
- Plugin
- Usage
- Audit

Enterprise

- Governance
 - Security
 - Cost
 - Marketplace
-

123. Builder 개발 우선순위

Visual Builder는 다음 순서를 권장한다.

1. Form-based Agent Builder
2. Workflow Graph Viewer

3. Workflow Editor
4. Plugin Configuration
5. Knowledge Configuration
6. Advanced Governance UI

API Contract보다 먼저 복잡한 Drag-and-Drop Builder에 과투자하지 않는다.

124. Documentation Development

문서는 코드 완료 후 별도로 작성하지 않는다.

주요 문서:

- API
- SDK
- Plugin
- Deployment
- Operations
- Administrator
- Developer

각 Release와 함께 갱신한다.

125. Development Risk

주요 개발 Risk:

- Scope Explosion
- Premature Microservices
- Model Provider Coupling
- Weak Runtime Durability
- Tenant Isolation Bug

- Schema Drift
 - Plugin Security
 - AI Evaluation Ambiguity
 - Cost Explosion
 - Too-Early Marketplace
-

126. Risk 대응

Scope Explosion

Stage Gate로 제어.

Microservice 과도화

Modular Monolith 우선.

Provider Coupling

Model Gateway 강제.

Runtime Risk

Durable State와 Failure Test 우선.

Tenant Risk

Cross-Tenant 자동 테스트 필수.

Plugin Risk

Sandbox와 Manifest.

Cost Risk

Usage 측정부터 선행.

127. Development KPI

초기 개발 과정에서 다음 KPI를 사용할 수 있다.

Engineering

- Build Success Rate
- Test Pass Rate
- Deployment Success
- Change Failure Rate
- Recovery Time

Platform

- Execution Success
- Queue Delay
- Runtime Recovery
- API Error Rate
- Tenant Isolation Incidents

AI

- Agent Task Success
- Workflow Completion
- Retrieval Quality
- Model Failure Rate

128. Milestone 완료 기준

Milestone은 다음이 충족되어야 완료한다.

- Feature complete
- Test complete
- Observability complete
- Documentation complete
- Migration complete
- Security review complete

- Acceptance complete

129. Development Phase Promotion 기준

다음 Phase로 이동하려면 기술적으로 다음을 확인한다.

Foundation Stable?

→ Yes

Previous Core Contract Stable?

→ Yes

Critical Bugs Controlled?

→ Yes

Security Gate Passed?

→ Yes

Operations Possible?

→ Yes

130. Development Roadmap Acceptance Criteria

본 Development Roadmap은 다음을 충족해야 한다.

1. Product Roadmap을 실제 개발 단계로 변환한다.
2. Core Platform이 Runtime보다 선행한다.
3. Runtime이 Agent 고도화보다 선행한다.
4. Agent와 Workflow가 Knowledge·Memory·Plugin보다 핵심 선행 구조를 갖는다.

5. Plugin Contract가 Marketplace보다 선행한다.
6. Governance가 L4/L5 이전에 구현된다.
7. Security와 Multi-Tenant가 전 단계에서 병행된다.
8. Observability가 초기부터 포함된다.
9. API와 Schema가 구현보다 먼저 정의된다.
10. Durable Execution이 명확한 핵심 단계로 포함된다.
11. Test와 Evaluation이 구분된다.
12. CI/CD가 초기 Foundation에 포함된다.
13. Dedicated Tenant와 Private Deployment가 Enterprise 단계에 포함된다.
14. Marketplace는 안정된 Package Contract 위에 구축된다.
15. AI 자율성은 단계적으로 확대된다.
16. 각 Phase의 완료 Gate가 존재한다.
17. Workstream 병렬화가 가능한 구조다.
18. 기술 부채 관리가 포함된다.
19. Release 및 Migration 전략이 포함된다.
20. WBS로 직접 세분화 가능한 구조다.

131. Development Roadmap 금지사항

다음 개발 방식은 허용하지 않는다.

- Runtime 없이 Agent 기능을 직접 서비스 코드에 구현
- Tenant Isolation을 개발 후반에 추가
- Provider SDK를 Agent 코드에서 직접 호출
- Schema 없이 API 구현부터 시작
- Workflow 상태를 Worker Memory에만 저장
- 모든 Domain을 처음부터 Microservice로 분리

- Plugin 보안을 Marketplace 출시 후 추가
- Test 없이 Feature 완료 처리
- Monitoring 없는 Production Release
- AI Evaluation 없이 Agent 품질을 수동 판단만으로 운영
- Database Table을 곧바로 Domain API로 노출
- Production 코드에 Secret Hard Coding
- Breaking Schema 변경을 Silent 적용
- Long-running Execution을 단일 HTTP Request로 처리
- L4/L5를 Governance 이전에 구현
- Marketplace를 Plugin Framework보다 먼저 개발
- UI를 API보다 우선해 Platform Contract를 UI에 종속
- 비용 계측 없이 자동화 기능 확대
- Rollback 전략 없는 Major Release
- Security Exception을 코드 우회로 구현

132. Development Roadmap의 최종 정의

AGEX Development Roadmap은 AGEX Product Roadmap에서 정의한 제품 발전 방향을 실제 개발 가능한 기술 단계와 Workstream으로 분해한 공식 실행 기준이다.

AGEX 개발은 Agent 기능부터 시작하지 않는다.

먼저 Tenant, Resource, Version, Policy, API 및 Audit을 포함한 Core Platform을 구축하고, 그 위에 Durable Runtime과 Execution State를 구축해야 한다.

Runtime 안정성이 확보된 이후 Agent와 Workflow를 구현하며, 이후 Knowledge, Memory 및 Plugin을 통합한다.

Enterprise 단계에서는 Governance, Advanced Security, Dedicated Tenant, Private Deployment 및 Cost Control을 추가한다.

Marketplace는 Plugin 및 Package Contract가 충분히 안정화된 이후 구축하며, 높은 수준의 L4/L5 Autonomous Agent는 Runtime, Governance, Security 및 Evaluation 체계가 운

영 수준으로 검증된 이후에만 개발한다.

모든 개발 단계에서 Tenant Isolation, Security, Observability, Testing 및 Documentation 은 별도 후속 작업이 아니라 기능 완료 조건에 포함한다.

AGEX Development Roadmap은 다음 문장으로 최종 정의한다.

AGEX Development Roadmap은 범용 AI Operating System을 Core Platform → Durable Runtime → Agent/Workflow → Knowledge/Memory/Plugin → Enterprise Control → Ecosystem → Autonomous Platform의 순서로 구현하고, 각 단계의 계약·보안·테스트·운영 기준을 충족한 경우에만 다음 단계로 확장하도록 규정하는 공식 개발 실행 로드맵이다.

본 문서는 AGEX의 개발 순서, 기술 의존성, Workstream, Release Gate 및 품질 기준을 규정하는 공식 Development Roadmap 문서로 사용한다.